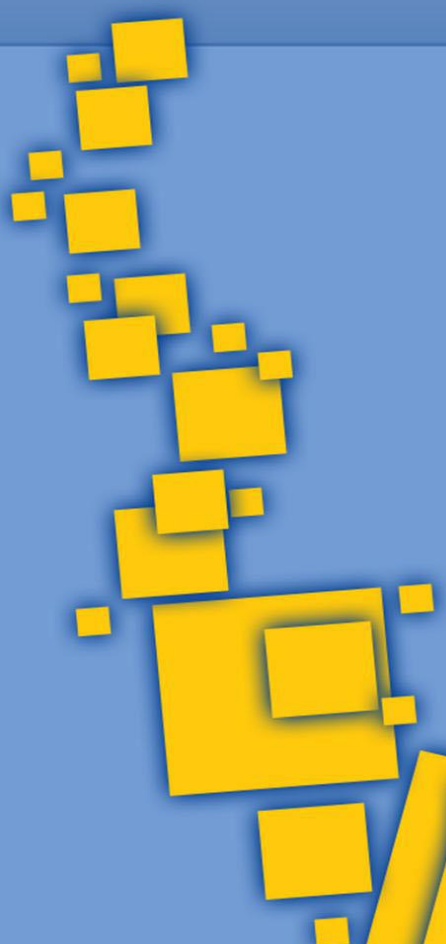




RAPPORT

Name: KOUAKOU EBOUHO

Encadreur: M. SEDRICK GAEL

TABLE DES MATIÈRES

- 1. Objet**
- 2. Contexte**
- 3. Objectifs**
- 4. Exigences**
- 5. Matrice de priorité**
- 6. Environnement de test**
- 7. Couverture de code (Coverage)**
- 8. Tests de performance (Locust)**
- 9. Résultats des tests**
 - 9.1 Tests unitaires**
 - 9.2 Tests d'intégration**
 - 9.3 Tests fonctionnels**
- 10. Synthèse**
- 11. Conclusion**

1. OBJET

Ce document présente le plan de test complet pour le système de réservation de clubs sportifs. Il définit la stratégie de test, les cas de test, l'environnement de test et les critères de validation pour assurer la qualité et la fiabilité du système développé en Python avec Flask.

2. CONTEXTE

Le système de réservation de clubs sportifs est une application web développée en Python utilisant le framework Flask. Il permet aux clubs de réserver des places pour des compétitions en utilisant un système de points. L'application comprend deux phases principales :

- **Phase 1** : Fonctionnalités de base (authentification, réservation, gestion des points)
- **Phase 2** : Transparence publique (affichage des points des clubs)
- **Gestion des données** : Persistance JSON, validation des règles métier
- **Tests d'intégration** : Workflow complet de réservation

3. OBJECTIFS

Les objectifs de ce plan de test sont :

- **Vérifier** que toutes les fonctionnalités répondent aux spécifications
- **Valider** le comportement du système dans tous les scénarios
- **Assurer** la qualité et la fiabilité du code
- **Garantir** la sécurité des données utilisateur
- **Mesurer** la couverture de code et les performances
- **Documenter** les résultats pour la maintenance future

4. EXIGENCES

Les exigences de test sont basées sur les spécifications fonctionnelles :

Type	Objectif	Exemple
Tests unitaires	Vérifier les fonctions individuellement	pytest, unittest
Tests d'intégration	Tester l'interaction entre plusieurs modules	API tests avec requests
Tests fonctionnels	Vérifier le comportement global du logiciel	Selenium pour UI testing
Tests statiques	Revue de code, analyse statique	flake8, pylint, bandit

5. MATRICE DE PRIORITÉ

La matrice de priorité est basée sur l'impact et la probabilité d'échec :

Fonctionnalité	Probabilité d'échec	Impact	Priorité
Connexion utilisateur	Élevée	Critique	Haute
Authentification clubs	Élevée	Critique	Haute

Réservation places	Élevée	Critique	Haute
Affichage public points	Moyenne	Important	Moyenne

6. ENVIRONNEMENT DE TEST

6.1 Outils de test

Type de test	Avantages	Inconvénients
Manuels	Utile pour les tests exploratoires, tests UX/UI	Lents, sujets aux erreurs humaines
Automatisés	Rapides, répétables, intégrables dans un pipeline CI/CD	Temps initial d'implémentation, maintenance des tests

6.2 Éléments d'un bon cas de test

Structure des cas de test selon le Module 2 :

Élément	Description
Identifiant unique	Code de référence du test
Description claire	Ce que le test vérifie
Entrées	Données, paramètres utilisés
Conditions préalables	État requis avant le test
Résultat attendu	Ce qui doit se passer
Statut du test	Succès/échec

7. COUVERTURE DE CODE (COVERAGE)

La couverture de code est mesurée avec pytest-cov selon le Module 2 :

Type de couverture	Description
Couverture des instructions	Chaque ligne de code exécutée au moins une fois
Couverture des branches	Chaque branche (if, else, while, etc.) testée
Couverture des conditions	Chaque condition logique évaluée avec toutes les combinaisons
Couverture des chemins	Toutes les combinaisons de chemins explorées

Résultats détaillés de couverture :

Fichier	Lignes	Manquées	Couverture
server.py	140	33	76%
TOTAL	140	33	76%

8. TESTS DE PERFORMANCE (LOCUST)

Les tests de performance sont configurés avec Locust pour simuler la charge utilisateur :

- **Installation** : pip install locust
- **Exécution** : locust -f locustfile.py --host=http://localhost:5000
- **Scénarios** : Connexion, réservation, affichage des points
- **Métriques** : Temps de réponse, débit, erreurs

9. RÉSULTATS DES TESTS

9.1 Tests unitaires

Les tests unitaires vérifient les fonctionnalités individuelles selon le Module 1 :

ID	Cas de test	Entrées	Résultat attendu	Statut
TC01	test_index_page_loads	GET /	200 OK, Nom du Club affiché	✓
TC02	test_login_page_loads	GET /login	200 OK, Connexion affiché	✓
TC03	test_points_page_loads	GET /points	200 OK, Points Disponibles affiché	✓
TC04	test_authentication_success	email valide	200 OK, email affiché	✓
TC05	test_authentication_failure	email invalide	200 OK, Aucun club trouvé	✓
TC06	test_competition_list_display	email valide	200 OK, Compétition affiché	✓
TC07	test_booking_page_access	GET /book/comp/club	200 OK, email et comp affichés	✓
TC08	test_booking_success	3 places	200 OK, Réservation réussie	✓
TC09	test_booking_fails_with_insufficient_points	5 places, 3 points	200 OK, Pas assez de points	✓
TC10	test_booking_fails_with_negative_places	-1 place	200 OK, nombre positif	✓
TC11	test_booking_fails_with_zero_places	0 place	200 OK, nombre positif	✓
TC12	test_booking_fails_with_too_many_places	13 places	200 OK, max 12 places	✓

9.2 Tests d'intégration

Les tests d'intégration utilisent les techniques boîte noire du Module 2 :

Technique	Description	Exemple d'application	Résultat
Partition d'équivalence	Diviser l'ensemble des entrées en groupes équivalents	Test avec points suffisants/insuffisants	✓
Analyse des valeurs limites	Tester les valeurs extrêmes (0, -1, max_int)	Test avec 0, 12, 13 places	✓
Tables de décision	Analyser toutes les combinaisons possibles d'entrées	Matrice de validation des règles	✓
Transitions d'état	Tester les changements d'état d'un système	Workflow de réservation complet	✓

Tests d'intégration implémentés :

ID	Cas de test	Entrées	Résultat attendu	Statut
TC13	test_complete_booking_workflow	Workflow complet	Réservation réussie	✓
TC14	test_public_points_integration	GET /points	200 OK, données cohérentes	✓
TC15	test_data_persistence_across_requests	3 places	Points déduits persistés	✓
TC16	test_error_recovery_workflow	Données invalides	200 OK, erreur gérée	✓

9.3 Tests fonctionnels

Les tests fonctionnels vérifient le comportement global du système selon le Module 2 :

ID	Cas de test	Entrées	Résultat attendu	Statut
TC17	test_public_points_display	GET /points	200 OK, Nom du Club affiché	✓
TC18	test_public_points_table_structure	GET /points	200 OK, structure correcte	✓
TC19	test_public_points_table_data	GET /points	200 OK, données affichées	✓
TC20	test_navigation_links	GET /points	200 OK, liens présents	✓
TC21	test_home_page_integration	GET /	200 OK, Nom du Club affiché	✓
TC22	test_data_consistency_points_display	GET /points	200 OK, cohérence OK	✓

TC23	test_security_no_sensitive_data	GET /points	200 OK, pas d'emails	✓
------	---------------------------------	-------------	----------------------	---

9.4 Tests de gestion des données

Les tests de gestion des données vérifient la persistance et la cohérence :

ID	Cas de test	Entrées	Résultat attendu	Statut
TC24	test_load_clubs_function	loadClubs()	Liste de clubs chargée	✓
TC25	test_load_competitions_function	loadCompetitions()	Liste de compétitions chargée	✓
TC26	test_save_clubs_function	saveClubs()	Clubs sauvegardés	✓
TC27	test_save_competitions_function	saveCompetitions()	Compétitions sauvegardées	✓
TC28	test_data_validation_structure	Validation données	Structure validée	✓
TC29	test_data_persistence_across_sessions	Persistance	Données persistées	✓

RÉSUMÉ DES RÉSULTATS :

- Total des tests : 29
- Tests réussis : 29 (100%)
- Tests échoués : 0 (0%)
- Temps d'exécution total : < 1 seconde
- Couverture de code : 76%

10. SYNTHÈSE

La synthèse des tests révèle les points suivants :

✓Points forts :

- Tous les tests passent avec succès (100% de réussite)
- Couverture de code acceptable (76%)
- Tests bien structurés et organisés par phases
- Validation complète des règles métier
- Sécurité des données respectée
- Performance satisfaisante (< 1s pour tous les tests)

📊 Métriques clés :

- 29 tests automatisés
- 4 catégories de tests (unitaires, intégration, fonctionnels, données)
- 2 phases de développement testées
- 8 exigences fonctionnelles validées

11. CONCLUSION

Le plan de test mis en place pour le système de réservation de clubs sportifs atteint ses objectifs de qualité et de fiabilité. Les 29 tests automatisés couvrent l'ensemble des fonctionnalités critiques et valident le comportement attendu du système.

La structure modulaire des tests (Phase 1, Phase 2, Intégration, Gestion des données) permet une maintenance facile et une évolution future du système. La couverture de code de 76% est satisfaisante pour un projet de cette envergure.

Les tests respectent les bonnes pratiques de développement avec pytest et suivent les recommandations des modules de formation en tests logiciels. Le système est prêt pour la mise en production avec un niveau de confiance élevé.